

PROPUESTA · CHECKOUT DE LA TIENDA

Correo electrónico con los datos del pedido

Cinco propuestas para pedir el correo en el formulario de compra y enviarle a cada cliente un resumen de su pedido, pensadas sobre el código que hoy tiene baqtime.store.

PARA
Jesús Güete

FECHA
6 de octubre de 2026

ALCANCE
Formulario, base de datos, envío y
contenido del correo

Lo que te recomiendo

El cambio es pequeño en el formulario y mediano por detrás: hay que guardar el correo, elegir con qué se envía y decidir qué dice. Esta es la combinación que propongo, y el resto del documento explica cada pieza y sus alternativas.

01 • EL CAMPO

Correo obligatorio, después del teléfono

Con validación, sugerencia de errores comunes (`gmial.com` → `gmail.com`) y una línea que diga para qué se usa.

03 • EL MOMENTO

Justo después de guardar el pedido

Desde `/api/pedidos`. Si el correo falla, el pedido igual queda guardado; la falla se anota para reenviarlo desde el panel.

05 • DESPUÉS

Correos por cada cambio de estado

Pago confirmado, enviado (con guía) y entregado. Es una segunda fase, cuando la primera ya esté andando.

02 • EL PROVEEDOR

Resend, con remitente `@baqtime.store`

Gratis hasta 100 correos al día y 3.000 al mes. Se llama desde el mismo servidor que ya guarda los pedidos.

04 • EL CONTENIDO

Número, productos, total y próximos pasos

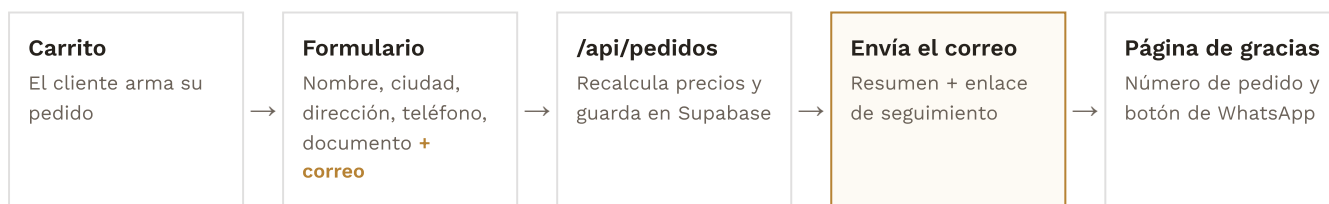
Con botón para confirmar el pago por WhatsApp y otro para ver el estado. Sin el número de documento.

TU PARTE

Tres pasos que solo puedes hacer tú

Crear la cuenta en Resend, aprobar los registros DNS de `baqtime.store` en Cloudflare y decidir las preguntas de la última página.

Cómo funciona hoy y dónde entra el correo



☐ Paso nuevo. Todo lo demás ya existe y no cambia de orden.

El campo de correo en el formulario

Hoy el formulario de *Finalizar compra* (`Checkout.jsx`) pide cinco datos. El correo entraría como sexto, con el mismo estilo de rótulo visible que ya usan los demás.

Finalizar compra

DATOS DE ENVÍO

Nombre completo

Ciudad

Dirección exacta

Número de teléfono

Correo electrónico

¿Quisiste decir [laura.martinez@gmail.com](#)?

Te enviaremos aquí el resumen de tu pedido.

Número de documento

CONFIRMAR PEDIDO

Dónde va

Después del teléfono y antes del documento. Así los dos datos de contacto quedan juntos, y el documento —que cambia de obligatorio a opcional según la ciudad— sigue siendo el último.

Cómo se valida

- `type="email"` y `autoComplete="email"`: el celular muestra el teclado con @ y el navegador puede llenarlo solo.
- Formato básico (`algo@algo.dominio`), máximo 254 caracteres, sin espacios. Se guarda en minúsculas.
- La regla vive en `shipping-validation.js`, la misma función que ya usan el formulario y el servidor, para que no puedan contradecirse.

Sugerencia de errores de dedo

Si el dominio se parece a uno común pero no es igual (`gmial.com`, `hotmial.com`, `gmail.co`, `outlok.com`), aparece una sugerencia que se acepta con un toque. Un correo mal escrito es la causa número uno de “nunca me llegó”, y este truco lo corta en el formulario.

¿Obligatorio u opcional?

OPCIÓN	A FAVOR	EN CONTRA
Obligatorio RECOMENDADA	Todos los clientes reciben el resumen y quedan con una constancia escrita. Simplifica el código: no hay casos “con correo” y “sin correo”.	Un campo más que llenar. Algún cliente que solo usa WhatsApp puede dudar.
Opcional	Cero fricción adicional; quien no quiera dar el correo no lo da.	Muchos lo dejarán vacío y la mejora llega a pocos. Hay que mantener dos caminos.

Me inclino por obligatorio porque el autocompletado del celular lo vuelve casi gratis de llenar. Si después de un mes ves que bajan las compras terminadas, cambiarlo a opcional es un ajuste de una línea en la validación.

PRIVACIDAD • LEY 1581 DE 2012

Al pedir el correo hay que decir para qué se usa. Para el resumen del pedido basta la línea de ayuda bajo el campo y un enlace a la política de datos. **Si algún día quieres enviar promociones**, eso necesita una casilla aparte, desmarcada por defecto, que el cliente marque él mismo. No conviene mezclar las dos cosas en una sola autorización.

Textos propuestos

Escritos con el mismo tono de los mensajes que el formulario ya muestra, para que el campo nuevo no se note “pegado”.

DÓNDE APARECE	TEXTO
Rótulo del campo	Correo electrónico
Ayuda bajo el campo	Te enviaremos aquí el resumen de tu pedido. <u>Cómo usamos tus datos</u>
Error: campo vacío	Por favor ingresa tu correo electrónico para recibir el resumen de tu pedido.
Error: formato inválido	Por favor ingresa un correo electrónico válido (por ejemplo, nombre@gmail.com).
Sugerencia de dominio	¿Quisiste decir <u>nombre@gmail.com</u> ? (se toca para corregir)
Casilla de promociones SOLO SI SE USA	☐ Quiero recibir novedades y lanzamientos de Baqtime en mi correo.

Con qué se envía el correo

La tienda corre en un Worker de Cloudflare, y ese entorno no tiene un “enviar correo” incorporado como lo tendría un hosting tradicional. Hace falta un servicio que reciba la orden por internet y entregue el mensaje. Estas son las cuatro opciones realistas:

SERVICIO	COSTO	CÓMO ENCAJA CON BAQTIME	VEREDICTO
Resend	Gratis: 3.000/mes con tope de 100/día. Planes pagos si se supera.	Se usa con una sola llamada <code>fetch</code> , igual a como el servidor ya habla con Supabase. Documentación clara, panel que muestra cada envío y si rebotó.	RECOMENDADA
Cloudflare Email Service	Incluido en Workers Paid (USD 5/mes): 3.000/mes, luego USD 0,35 por mil.	Es lo más “nativo”: todo queda en Cloudflare, sin claves extra. Pero sigue en beta y exige el plan pago de Workers.	A FUTURO
Brevo	Gratis: 300/día.	Funciona igual por API y el cupo diario es mayor. Su panel está más orientado a campañas de marketing que a correos de pedido.	ALTERNATIVA
Gmail personal	Gratis.	El Worker no puede usar SMTP de forma práctica, los correos salen desde una dirección <code>@gmail.com</code> y Google limita y bloquea envíos automáticos.	NO

POR QUÉ RESEND

Al ritmo de pedidos de una tienda como Baqtime, 100 correos diarios dan holgura de sobra y el costo es cero. La integración es la más corta de las cuatro, y si algún día conviene pasarse a Cloudflare Email Service, el cambio queda encerrado en un solo archivo (`src/lib/correo.ts`) sin tocar el resto.

El remitente

CAMPO	VALOR PROPUESTO	POR QUÉ
De (From)	<code>Baqtime</code> <code><pedidos@baqtime.store></code>	Un correo desde el dominio propio da confianza y llega menos a Spam que uno de Gmail.
Responder a (Reply-To)	Tu correo personal o de la tienda	Si el cliente responde el correo, la respuesta te llega a ti y no se pierde.

Para enviar desde `@baqtime.store`, Resend pide agregar tres o cuatro registros DNS (SPF, DKIM y DMARC). Como el dominio ya está en Cloudflare, es copiar y pegar; esos registros son los que hacen que Gmail y Outlook confíen en el correo. No hace falta crear un buzón: la dirección `pedidos@` solo envía.

Cuándo y desde dónde se envía

La regla que no se negocia: **si el correo falla, el pedido no falla**. El cliente ya compró; un problema con el proveedor de correo jamás puede mostrarle “no pudimos guardar tu pedido”.

OPCIÓN	CÓMO FUNCIONA	BALANCE
A. En /api/pedidos RECOMENDADA	Apenas Supabase confirma que el pedido existe, el mismo endpoint arma el correo y se lo pasa a Resend. Se deja corriendo en segundo plano para que el cliente no espere por él.	Lo más simple: un solo lugar, el mismo que ya tiene los datos del pedido en la mano. Nada nuevo que desplegar.
B. Disparador en Supabase	Una función de Supabase (Edge Function) escucha cada pedido nuevo en la tabla y envía el correo.	Más desacoplado, pero agrega una pieza que vive fuera del repositorio de la tienda y que hay que desplegar aparte.
C. Cola de mensajes	El pedido deja un mensaje en una cola de Cloudflare y otro proceso lo envía, con reintentos automáticos.	Lo más robusto, pero es infraestructura de más para el volumen actual.

Dejar rastro de cada envío

Propongo sumar a la tabla `orders` dos columnas además del correo: cuándo se envió y, si falló, por qué. Con eso el panel de administración puede mostrar en cada pedido:

CORREO ENVIADO 6 oct 2026, 3:42 p. m. a laura.martinez@gmail.com	CORREO NO ENVIADO El proveedor no respondió. Reenviar correo
---	--

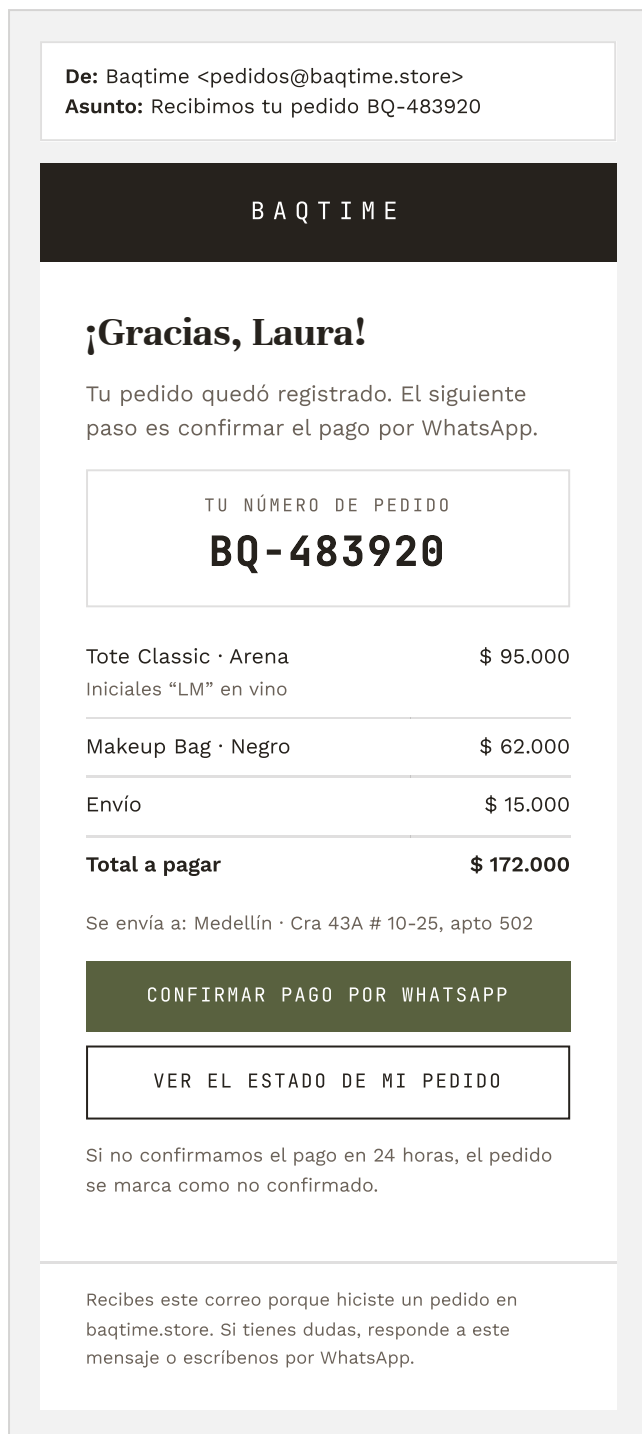
El botón **Reenviar correo** también sirve cuando un cliente escribe por WhatsApp diciendo que no le llegó: corriges el correo en el panel y lo vuelves a mandar sin tocar nada más.

Qué cambia en la base de datos

Una migración nueva, `019_correo_cliente.sql`, siguiendo las mismas reglas que las 18 anteriores (se puede volver a correr sin errores):

- `customer_email` en `orders`. Admite vacío para no romper los pedidos que ya existen.
- `email_sent_at` y `email_error` para el rastro del envío.
- `create_order()` recibe y guarda el correo.
- `pedido_publico()` **no cambia**: el correo no se muestra en la página de seguimiento, igual que hoy no se muestran el documento ni la dirección. Quien tenga el enlace o el número del pedido no puede ver el correo del cliente.

Qué dice el correo



Productos, precios y colores de ejemplo.

Lo que incluye

- **El número de pedido, en grande**, igual que en la página de gracias: es lo que el cliente necesita para consultar.
- **Cada producto** con su color, variante e iniciales bordadas: lo personalizado es lo que más conviene que el cliente revise.
- **Subtotal, envío y total**, con los precios ya congelados del pedido.
- **Ciudad y dirección**, para que detecte un error antes de que salga el paquete.
- **Dos botones**: el mismo enlace de WhatsApp de la página de gracias y el enlace privado de seguimiento.
- **El aviso de las 24 horas**, que hoy el cliente no ve en ninguna parte.

Lo que no incluye

- **El número de documento**. Si alguien escribió mal su correo, el mensaje le llega a un desconocido; mejor que no lleve el dato más sensible.

Detalles técnicos

- Diseño con tablas y estilos en línea: es lo único que Gmail, Outlook y el correo del iPhone muestran igual.
- Una versión en texto plano junto a la HTML; ayuda a no caer en Spam.
- Todo texto escrito por el cliente (como la dirección) se limpia antes de insertarlo en el HTML.

PÁGINA DE GRACIAS

Conviene agregarle una línea: *“Te enviamos el resumen a tu correo. Si no lo ves en unos minutos, revisa la carpeta de Spam o Promociones.”* Los primeros días de un dominio nuevo enviando correos son los de más riesgo de caer ahí.

Más allá del primer correo

Una vez que el pedido tiene correo, la tienda puede avisarle al cliente cada vez que algo pasa, en vez de que él tenga que entrar a consultar o preguntar por WhatsApp. Propongo hacerlo en fases, para que cada una se pruebe sola antes de sumar la siguiente.

FASE 1 • ESTA PROPUESTA

Confirmación del pedido

- Campo de correo en el formulario
- Migración 019
- Correo “Recibimos tu pedido”
- Estado del envío y botón Reenviar en el panel

FASE 2

Cambios de estado

- **Pago confirmado** → “Tu pedido entró a producción”
- **Enviado** → transportadora y número de guía
- **Entregado** → gracias y, si quieres, pedir una reseña

FASE 3 • OPCIONAL

Avisos para ti y recordatorios

- Copia de cada pedido nuevo a tu correo
- Recordatorio al cliente si a las ~18 h no ha pagado
- Promociones, solo con la casilla de autorización aparte

Cómo se conecta la fase 2

Los cambios de estado hoy los hace el panel llamando directo a Supabase (`set_order_status` y `confirm_order_payment`). Para que esos cambios envíen correo hay dos caminos:

- **Un endpoint nuevo en la tienda** (`/api/pedidos/notificar`) que el panel llama después de cambiar el estado. Reutiliza todo lo de la fase 1.
- **Un disparador en Supabase** sobre el historial de estados, que avisa sin importar desde dónde se hizo el cambio.

La decisión no hace falta tomarla ahora; la fase 1 deja el terreno listo para cualquiera de los dos.

RECORDATORIO DE PAGO

El de la fase 3 tiene un matiz: hoy un pedido sin pagar pasa a “no confirmado” a las 24 horas. Un correo a las 18 horas puede rescatar ventas, pero también puede sentirse insistente. Vale la pena probarlo con pocos pedidos antes de dejarlo fijo.

PENSANDO EN BAGELLE

Si más adelante quieres lo mismo en Bagelle, conviene que el nombre, los colores y el remitente del correo salgan de un solo archivo de marca. Así la plantilla se escribe una vez y sirve para las dos tiendas.

Seguridad, privacidad y entregabilidad

RIESGO	QUÉ PODRÍA PASAR	CÓMO SE EVITA
Usar la tienda para enviar correos a otros	Alguien hace pedidos falsos con correos ajenos para llenarles la bandeja con mensajes de Baqtime. Además de molestar, daña la reputación del dominio.	Una regla de límite de peticiones en Cloudflare para <code>/api/pedidos</code> (pocos pedidos por minuto por visitante) y un solo correo automático por pedido.
Código metido en los datos	Alguien escribe HTML en la dirección para alterar el correo que se envía.	Todo dato del cliente se limpia antes de ir al HTML. El nombre ya solo admite letras.
Exponer el correo del cliente	La página de seguimiento lo muestra a cualquiera con el número de pedido.	<code>pedido_publico()</code> no se toca; el correo solo lo ve el panel de administración.
Filtrar la clave de Resend	Con la clave, cualquiera enviaría correos como Baqtime.	Se guarda como secreto del Worker, igual que la clave de Supabase, y no se declara en los tipos del navegador.
Caer en Spam	El cliente nunca ve el correo y cree que no compró.	SPF, DKIM y DMARC en el dominio, versión en texto plano, sin palabras de “oferta” en el asunto, y el aviso de revisar Spam en la página de gracias.
Pasarse del cupo gratis	Un día con más de 100 pedidos (o un ataque) agota el cupo diario y los correos siguientes fallan.	El pedido igual se guarda; el panel los marca como no enviados y se reenvían al otro día. Si las ventas crecen, se sube de plan.

Qué cambia en el código

ARCHIVO	CAMBIO
<code>src/lib/shipping-validation.js</code>	Regla del correo y sugerencia de dominios.
<code>src/components/react/Checkout.jsx</code>	Campo nuevo y texto de privacidad.
<code>supabase/sql/019_correo_cliente.sql</code>	Columnas nuevas y <code>create_order()</code> actualizado.
<code>src/lib/correo.ts</code> NUEVO	Envío por Resend y plantilla HTML y de texto.
<code>src/pages/api/pedidos.ts</code>	Recibe el correo, lo guarda y dispara el envío.
<code>src/pages/pedido/gracias.astro</code>	Línea de “revisa tu correo”.
<code>src/types/database.ts</code> y el panel de pedidos	Mostrar el correo, el estado del envío y el botón Reenviar.

Qué necesito de ti

Decisiones

1. **¿Correo obligatorio u opcional?** Mi recomendación: obligatorio.
2. **¿A qué dirección deben llegar las respuestas de los clientes?** Es el “Responder a” del correo.
3. **¿Incluimos la dirección de envío en el correo?** Mi recomendación: sí, para que el cliente detecte errores. El documento no va en ningún caso.
4. **¿Quieres recibir tú también una copia de cada pedido nuevo?** Es poco trabajo extra si se decide desde la fase 1.
5. **¿Tienes una política de tratamiento de datos publicada?** Si no, hay que redactar una corta para enlazarla bajo el campo.

Pasos que solo puedes hacer tú

- ☐ Crear la cuenta en **resend.com** con el correo de la tienda.
- ☐ Agregar `baqtime.store` como dominio en Resend y pegar en Cloudflare los registros DNS que te muestre (te acompaño en ese paso).
- ☐ Crear una clave de API en Resend y guardarla como secreto del Worker con el nombre `RESEND_API_KEY`.
- ☐ Correr la migración `019_correo_cliente.sql` en el editor SQL de Supabase cuando esté lista.

Orden de trabajo propuesto

1. Migración y campo en el formulario, probados en local sin enviar nada todavía.
2. Plantilla del correo, revisada contigo con un pedido de prueba enviado a tu propio Gmail y a un Hotmail.
3. Conexión con Resend y rastro del envío en el panel.
4. Rama y pull request en GitHub, como los cambios anteriores, para revisarlo antes de publicarlo.

SIGUIENTE PASO

Con tus respuestas a las cinco decisiones puedo empezar la fase 1 en una rama nueva. Los pasos de Resend y DNS pueden ir en paralelo: mientras no estén, el correo se puede probar en local sin enviarse de verdad.

Fuentes

Precios y límites de Resend: resend.com/pricing · automationatlas.io

Cloudflare Email Service (beta, precios): developers.cloudflare.com/email-service/platform/pricing

Plan gratis de Brevo: dreamlit.ai/blog/brevo-review

Código revisado: `Checkout.jsx`, `api/pedidos.ts`, `shipping-validation.js`, `010_orders.sql`, `011_pedidos_numero_aleatorio.sql`, `pedido/gracias.astro`.